

NarrativeFlow: Flow-Based Vision-Language-Action Model Using Robot Velocity Fields

Shota Kobayashi, Koki Seno, Daichi Yashima, and Komei Sugiura

Keio University, Japan

{shotakoba10267,koki.seno,ydaichi1207,komei.sugiura}@keio.jp

Abstract. We focus on language-conditioned flow-based manipulation, where robot flows (robot velocity fields) serve as embodiment-agnostic, motion-centric representations for leveraging data collected from multiple robot platforms. This task is crucial because language-conditioned manipulation is essential for practical robotic systems, yet scaling robot foundation models remains limited by the labor-intensive collection of embodiment-specific data. Existing methods either coarsely approximate robot flows with sparse keypoint displacements, or cannot handle language-conditioned manipulation. To address this limitation, we propose NarrativeFlow, which models robot flows as continuous velocity fields using a flow-matching formulation conditioned on language. Accordingly, NarrativeFlow generates robot flows that are physically consistent with real-world manipulation. To validate NarrativeFlow, we have conducted experiments on standard datasets for language-conditioned manipulation. The experimental results show that NarrativeFlow outperforms representative baseline methods on standard evaluation metrics. Furthermore, through real-world experiments, we show that NarrativeFlow achieves higher success rates than baseline methods across multiple manipulation tasks. The project page is available at <https://shota0520.github.io/NarrativeFlow-project-page/>

Keywords: Learning from cross-embodiment data · Flow matching · Language-conditioned manipulation

1 Introduction

Labor shortages have increased demand for robotic systems that support workers and reduce physical burden in manufacturing, logistics, and healthcare. To meet this demand, such systems must perform diverse tasks across a wide range of real-world environments, while following natural language instructions. In this direction, recent work has explored robot foundation models, which are often pretrained on large-scale data collected from multiple robot platforms [3,4,24,33]. This line of work indicates that the scale and diversity of training data are critical

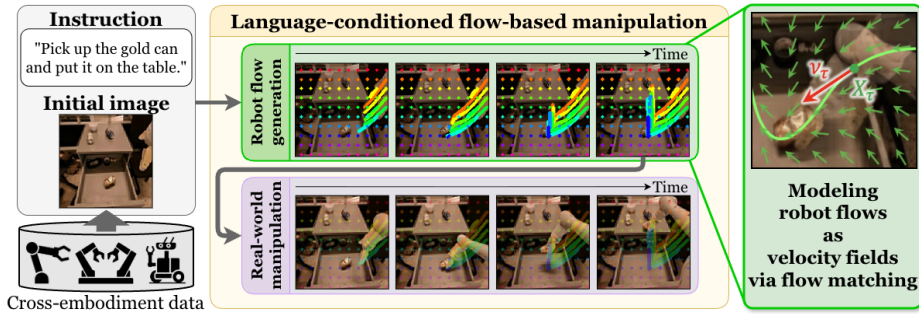


Fig. 1: Overview of our approach to language-conditioned flow-based manipulation. We formulate robot flows as continuous velocity fields via flow matching. For training, we use data collected from multiple robot platforms. At test time, given an instruction and an initial image, our model generates a robot flow. The robot then executes the object manipulation task based on the robot flow.

to model performance, yet training these models still requires labor-intensive collection of embodiment-specific data [3, 4, 24, 33].

In this study, we focus on language-conditioned flow-based manipulation. In this task, a model generates a robot flow (robot velocity field) conditioned on an instruction, and then a robot performs object manipulation based on the robot flow. Robot flows serve as embodiment-agnostic intermediate representations that bridge visual observations and robot actions. Because such flows can be extracted from arbitrary robot videos, flow-based approaches can leverage large-scale data collected from multiple robot platforms rather than relying on costly embodiment-specific data collection. However, this task remains challenging: indeed, a representative method, Im2Flow2Act [49], still underperforms a method using oracle robot flows by 21 percentage points in average success rate (see Table 4).

Many existing flow-based manipulation methods [2, 13, 49] do not formulate robot flows as dense velocity fields, but as displacements of sparse keypoints, even though robot manipulation is inherently a time-evolving physical process. Specifically, these methods predict sparse keypoint coordinates and then derive robot flows from their displacements, yielding only a coarse approximation of continuous robot motion. These formulations can be problematic because they assume that the displacements are deterministic, although such an assumption is rarely satisfied in practice. By contrast, Flow as Flow [38] can model robot flows as dense velocity fields; however, it cannot handle language-conditioned settings. Moreover, existing methods [2, 13, 38, 49] typically rely only on primary flow-generation objectives during training, which provide no explicit supervision for capturing task-relevant semantics. As a result, such models can be misled by task-irrelevant visual details, such as background appearance and lighting.

To address these limitations, we propose NarrativeFlow, a language-conditioned robot flow generation method that formulates robot flows as probability velocity

fields within a flow-matching framework. By modeling velocity fields, NarrativeFlow generates robot flows that are more physically consistent with real-world manipulation. Moreover, we introduce an auxiliary loss that aligns the model’s intermediate representations with task-relevant scene changes, enabling the model to capture task-relevant semantics.

Fig. 1 shows an overview of our approach. We first train our model on cross-embodiment datasets. At deployment, given an instruction and an initial image, the model generates a robot flow that represents the robot motion required for the specified task (e.g., picking up the can and placing it on the table). Subsequently, the robot executes the manipulation based on the generated flow to complete the task.

The contributions of this study are as follows:

1. We propose NarrativeFlow, a language-conditioned robot flow generation method that formulates robot flows as probability velocity fields within a flow-matching framework, enabling high-quality robot flow generation.
2. We introduce the Narrative Delta loss as an auxiliary objective that aligns the model’s intermediate representations with task-relevant scene changes. We further introduce two subtask tokens to mitigate interference between this auxiliary objective and the primary flow-generation objective.
3. We show that NarrativeFlow outperforms baseline methods across standard datasets.
4. Through real-world experiments, we show that NarrativeFlow achieves higher success rates than baseline methods across multiple manipulation tasks.

2 Related Work

Language-Conditioned Manipulation. Robotic foundation models aim to address diverse tasks across varied environments and have been actively investigated [22, 62]. Recent language-conditioned manipulation policies build on pre-trained foundation models such as multimodal large language models (MLLMs), adopting them as semantic backbones coupled with action heads that map language and observations to control [3, 4, 28]. This design inherits semantic and perceptual priors from large-scale pretraining, which transfer to unseen tasks and objects [3, 53].

A separate line of work introduces an explicit intermediate representation between language and action [8, 25, 49]. Such methods predict task- or scene-level targets, including 3D affordance maps (e.g., [18, 25]) and visual subgoals (e.g., [5, 61]), rather than conditioning the policies directly on latent features from the backbones. Among such targets, robot flows represent task progress as point or end-effector motion, bridging language specifications and the low-level trajectories that execute them [13, 19, 49, 52].

Flow-Based Manipulation. Flow-based manipulation methods use robot flows as embodiment-agnostic motion representations that bridge vision and action [2, 9, 11, 49, 58]. These representations can be extracted from heterogeneous robot

videos (e.g., [43, 47]) and from web-scale human videos (e.g., [2, 38, 56]). Moreover, a large body of work has demonstrated the effectiveness of robot flows across diverse real-world tasks including rigid, articulated, and deformable object manipulation (e.g., [49, 59]).

Although earlier methods generate robot flows over the entire frame (e.g., [2, 47]), some studies use object-centric or hand-centric flows to further reduce the influence of embodiment-specific factors (e.g., [11, 19, 49, 56]). Other approaches adopt flow-conditioned video generation to predict the future scene (e.g., [13]), or extend 2D flows into 3D using depth estimation (e.g., [9, 54]).

These methods derive robot flows from the displacements of sparse keypoints. As an alternative, Flow as Flow [38] directly predicts robot velocity fields via flow matching, a framework that predicts probability velocity fields in a generation space. However, this method is limited to goal-conditioned manipulation and hence it is unable to address language-conditioned settings. In contrast, NarrativeFlow generates robot flows using the flow-matching formulation conditioned on language.

Learning Scene-Change Representations. To model temporal dynamics and task-relevant transformations, prior work has leveraged scene-change information in visual representation learning. For example, procedure-planning methods predict sequences of high-level action labels from a current visual observation toward a visual goal in instructional video understanding [26, 31, 45]. Other work has focused on scene changes for object-centric localization (e.g., [51]), procedural modeling (e.g., [32]), and fine-grained temporal modeling (e.g., [27, 50]). In parallel, recent work has explored efficient temporal representations using video codecs, further leveraging temporal and scene-change cues (e.g., [55]).

In manipulation tasks, goal images are often used to specify desired final scene configurations [5, 40]. These images provide fine-grained details of the scene change to be achieved, but are typically unavailable at inference time in language-conditioned settings. To leverage this information for language-conditioned manipulation, GRIF [30] uses initial-goal image pairs during training to learn scene-change representations aligned with instructions.

In addition, world models and dynamics models predict how objects and scenes change conditioned on actions. They model such changes through future observation prediction, learned physical dynamics, or dense geometric motion [5, 12, 17].

3 Method

3.1 Problem Statement

In this study, we address language-conditioned flow-based manipulation. This task consists of two subtasks: robot flow generation and object manipulation conditioned on the generated robot flow. In the robot flow generation subtask, given an initial image and an instruction, the model generates a robot flow, which is expected to be aligned with the ground-truth robot motion. Subsequently, in the object manipulation subtask, at each timestep, given the current observation

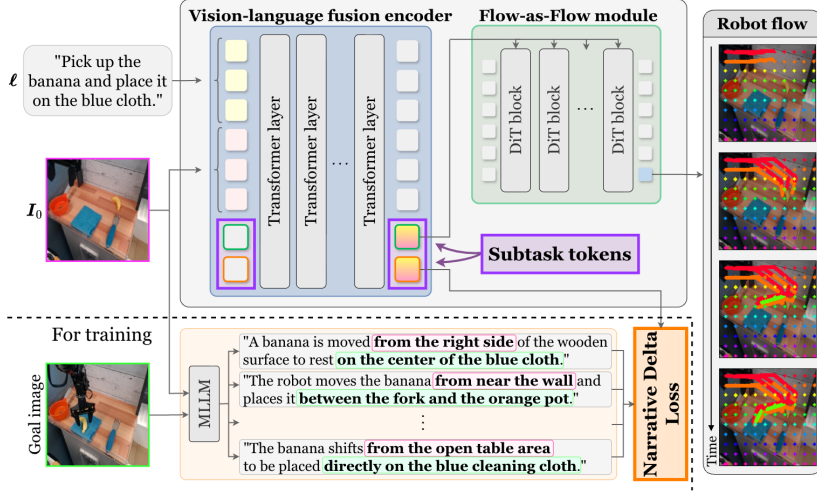


Fig. 2: Architecture of NarrativeFlow. The vision-language fusion encoder integrates the embeddings of ℓ and I_0 into the two subtask tokens: one for conditioning the Flow as Flow module to generate the robot flow, and the other for computing the Narrative Delta loss. During training, the Narrative Delta loss is used as an auxiliary training objective, aligning the corresponding subtask token with the narrative representations of task-relevant scene changes from I_0 to the goal image.

and the generated robot flow, the model outputs a robot end-effector pose. Fig. 1 shows an example of this task. Given the initial image and the instruction “Pick up the gold can, and place it on the table,” the model is required to generate the corresponding robot flow. Based on the generated robot flow, the robot then moves the gold can onto the table.

In this study, we assume that the end-effector is visible in an initial image under a fixed camera setup. We focus on the flow generation subtask. For the object manipulation subtask, we use a flow-conditioned manipulation policy, such as those adopted in prior methods (e.g., [2, 38, 49]).

3.2 Overview

We propose NarrativeFlow, a method for language-conditioned robot flow generation, inspired by flow-based manipulation policies [13, 38, 49]. Fig. 2 shows the architecture of NarrativeFlow. Our model consists of the vision-language fusion encoder and the Flow-as-Flow module. The model inputs are an instruction ℓ and an initial image I_0 , and they are encoded using Qwen3-Embedding-0.6B [60] and DINOv3-ViT-B/16 [39] to obtain language and visual embeddings z_{lang} and z_{img} , respectively.

3.3 Vision-Language Fusion Encoder

The vision-language fusion encoder integrates $\mathbf{z}_{\text{img}}$ and $\mathbf{z}_{\text{lang}}$ into two purpose-specific joint representations, which are used in two downstream paths: one for conditioning the Flow-as-Flow module (Sec. 3.5) and the other for computing the Narrative Delta loss (Sec. 3.4). A standard approach to aggregating multimodal inputs into a single representation is to add a learnable special token (e.g., a [CLS] token) to the inputs and use its output as a global readout [10, 41]. However, this single-token design can be insufficient for our model, in which the same token is used both to condition the Flow-as-Flow module and to compute the Narrative Delta loss. This can entangle the learning signals from both downstream paths, hindering the token from capturing purpose-specific information.

To address this issue, we introduce two subtask tokens, each dedicated to a distinct downstream path. This separation enables each token to act as a purpose-specific readout, helping to mitigate interference between the learning signals from the two downstream paths. Specifically, the subtask tokens are concatenated with $\mathbf{z}_{\text{img}}$ and $\mathbf{z}_{\text{lang}}$, and the resulting token sequence is processed by a Transformer encoder consisting of Transformer layers. The output tokens corresponding to the subtask tokens are projected into $\mathbf{z}_{\text{st}}^{(1)}$ and $\mathbf{z}_{\text{st}}^{(2)}$, which are used to condition the Flow-as-Flow module and to compute the Narrative Delta loss, respectively.

3.4 Narrative Delta Loss

We introduce the Narrative Delta loss, which is an auxiliary loss, to guide $\mathbf{z}_{\text{st}}^{(2)}$ toward narrative representations [14] of task-relevant scene changes. Existing robot flow generation methods [13, 38, 49] are typically trained using only a loss for flow generation (e.g., Eq. (5)). Although such a loss supervises the target robot flow, it does not explicitly constrain these methods’ intermediate representations, which may leave task-relevant scene-change information entangled with task-irrelevant visual details. Such entanglement can reduce robustness to nuisance variations such as background appearance or lighting, which often vary across practical environments. To mitigate this issue, we use auxiliary supervision constructed from textual descriptions of task-relevant scene changes from $\mathbf{I}_0$ to a goal image $\mathbf{G}$.

Specifically, we construct the auxiliary targets offline as follows: First, we provide an MLLM with $\mathbf{I}_0$, $\mathbf{G}$, and a prompt instructing it to describe task-relevant changes (e.g., robot pose changes and object displacements) while ignoring task-irrelevant visual details (see Supplementary Materials). The MLLM then generates N_c captions that describe the same manipulation from different perspectives. Next, we embed the generated captions using Qwen3-Embedding-0.6B to obtain the target narrative representations $\{\mathbf{z}_{\text{ND}}^{(i)}\}_{i=1}^{N_c}$. Finally, we compute the Narrative Delta loss in the narrative-representation space using $\mathbf{z}_{\text{st}}^{(2)}$:

$$\mathcal{L}_{\text{ND}} = \frac{1}{N_c} \sum_{i=1}^{N_c} \left[1 - \text{cossim} \left(\mathbf{z}_{\text{st}}^{(2)}, \mathbf{z}_{\text{ND}}^{(i)} \right) \right], \quad (1)$$

where $\text{cossim}(\cdot, \cdot)$ denotes cosine similarity. By aligning $\mathbf{z}_{\text{st}}^{(2)}$ with the multiple narrative representations, the Narrative Delta loss encourages $\mathbf{z}_{\text{st}}^{(2)}$ to capture scene changes that are consistently described across the generated captions. We use the Narrative Delta loss only during training and G is not required at inference time.

3.5 Flow-as-Flow Module

We use Flow as Flow [38] for generating a robot flow conditioned on $\mathbf{I}_0$ and $\mathbf{z}_{\text{st}}^{(1)}$. The original Flow as Flow method is designed for goal-conditioned manipulation and thus cannot address language-conditioned settings. This limits its applicability in practical settings, where $\mathbf{G}$ is often unavailable before manipulation, while ℓ is readily available. We therefore extend the method to language-conditioned settings using $\mathbf{z}_{\text{st}}^{(1)}$ as a language-conditioning signal.

In this module, we initialize N (e.g., 10×10) points uniformly on $\mathbf{I}_0$, and obtain their future positions by integrating velocity fields predicted by this module $\mathbf{v}_\theta$ at each timestep t . Let $\mathbf{\Xi}_t$ denote the ground-truth point coordinates at t , obtained by arranging the 2D coordinates of the N points into an $N \times 2$ matrix. The corresponding velocities are denoted as $\dot{\mathbf{\Xi}}_t$. We construct a target velocity field with a stabilization term as follows:

$$\mathbf{v}(\mathbf{\Xi}_t, \mathbf{X}, t) = \dot{\mathbf{\Xi}}_t - k(\mathbf{X} - \mathbf{\Xi}_t), \quad (2)$$

where $\mathbf{X} \sim \mathcal{N}(\mathbf{\Xi}_t, \sigma_0^2 e^{-2kt} \mathbf{I})$ with a small constant $\sigma_0 > 0$ and a stabilization coefficient k .

We implement $\mathbf{v}_\theta$ based on the Diffusion Transformer (DiT) architecture [34]. Each DiT block is modulated via adaLN-Zero [34] using a conditioning vector constructed from $\mathbf{z}_{\text{st}}^{(1)}$, $\mathbf{I}_0$, and t . Specifically, $\mathbf{I}_0$ and t are encoded using ResNet-18 [15] and sinusoidal positional encoding, respectively, and their resulting features are added to $\mathbf{z}_{\text{st}}^{(1)}$ to form the conditioning vector. At each timestep t , we form an input sequence $\mathcal{X}_{0:t} \in \mathbb{R}^{(t+1) \times N \times 2}$ by appending the current point coordinates $\mathcal{X}_t$ to the past point coordinate sequence $\mathcal{X}_{0:t-1}$. Then, the coordinates of the N points are flattened into a vector in $\mathbb{R}^{2N}$, which is used as a token. The resulting sequence is processed by the DiT blocks, and the last token represents the velocity field. For inference, the coordinates at t , denoted as $\mathbf{X}_t$, are obtained as follows:

$$\mathbf{X}_t = \mathbf{X}_0 + \int_0^t \mathbf{v}_\theta(\mathbf{X}_\tau, \tau \mid \mathbf{I}_0, \mathbf{z}_{\text{st}}^{(1)}, \mathcal{X}_{<\tau}) d\tau, \quad (3)$$

where $\mathcal{X}_{<\tau}$ denotes the history of the point coordinates at previous timesteps.

The overall loss $\mathcal{L}$ is defined as follows:

$$\mathcal{L} = \mathcal{L}_{\text{FaF}} + \lambda \mathcal{L}_{\text{ND}}, \quad (4)$$

where λ is a weighting hyperparameter, and $\mathcal{L}_{\text{FaF}}$ is the conditional flow matching loss [29, 38]:

$$\mathcal{L}_{\text{FaF}} = \mathbb{E}_{\mathbf{\Xi}_t, t, \mathbf{X}} \left[\left\| \mathbf{v}_\theta(\mathbf{X}, t \mid \mathbf{I}_0, \mathbf{z}_{\text{st}}^{(1)}, \mathbf{\Xi}_{0:t-1}) - \mathbf{v}(\mathbf{\Xi}_t, \mathbf{X}, t) \right\|^2 \right]. \quad (5)$$

Table 1: Statistics of the datasets used in our experiments.

Dataset	Embodiment	Number of samples	Avg. frame rate [fps]	Avg. duration [s]	Vocabulary size	Avg. sentence length [words]
Fractal [6]	Google Robot	87,212	3.00	14.15	50	5.70
Bridge V2 [44]	WidowX	60,064	5.00	7.51	3,012	8.05

4 Experiments

4.1 Experimental Setup

Datasets. We used video clips from Fractal [6] and Bridge V2 [44], both of which are standard real-robot datasets in robot learning [24,33]. Table 1 provides an overview of the datasets used in our experiments. The two datasets were collected from different real-world robotic platforms, leading to variation in the recorded motion and visual appearance of the scenes (e.g., backgrounds and viewpoints). For Fractal, the training, validation, and test sets contain 84,212, 1,000, and 2,000 samples, respectively. For Bridge V2, the corresponding sets contain 57,064, 1,000, and 2,000 samples, respectively. We used the training sets to train the model, the validation sets to tune the hyperparameters, and the test sets to evaluate the model.

Following prior work [38], we curated the datasets as described below. We set $\mathbf{I}_0$ as the first frame of the clip for each episode. In addition, we set $\mathbf{G}$ according to the binary success label provided for each episode. Concretely, for a successful episode, $\mathbf{G}$ was set as the frame at which success was determined, whereas for an unsuccessful episode, it was set as the last frame of the clip. We used ℓ corresponding to $\mathbf{I}_0$ when ℓ was included in the original datasets; otherwise, ℓ was generated by Qwen2.5-VL [1] from the given $\mathbf{I}_0$ and $\mathbf{G}$. To obtain $\Xi_{0:T-1}$, we used CoTracker3 [20], an off-the-shelf point-tracking method. Here, T denotes the sequence length of the robot flow. Specifically, we initialized N points uniformly on $\mathbf{I}_0$ and tracked their trajectories throughout the clip.

Following prior work [38], the datasets were preprocessed as described below. First, all images were resized to 256×256 pixels. Next, we obtained a segmentation mask of a robot end-effector from $\mathbf{I}_0$ for each video and incorporated it as an alpha channel of $\mathbf{I}_0$ in the model input. End-effector masks were generated using Robot-SAM [57]. Because the pretrained Robot-SAM often generated masks covering the entire robot arm, we fine-tuned Robot-SAM to localize only the end-effector. Specifically, we used 300 samples from the RoboSeg [57] dataset with additional end-effector mask annotations.

Baselines. We used representative language-conditioned robot flow generation baselines, FLIP [13] and Im2Flow2Act [49]. Both were trained on the same datasets as the proposed method.

Metrics. Following prior work [38], we used average displacement error (ADE), final displacement error (FDE), and less than delta ratio (LTDR) as evaluation metrics, with ADE as the primary metric. ADE was computed by averaging the

Table 2: Quantitative comparison between the proposed method and baseline methods. The best scores for each metric are shown in **bold**.

Method	Fractal			Bridge V2		
	ADE ↓	FDE ↓	LTDR ↑[%]	ADE ↓	FDE ↓	LTDR ↑[%]
FLIP [13]	66.17	87.52	35.69	50.73	68.43	47.72
Im2Flow2Act [49]	37.14	47.74	60.61	51.48	70.93	47.97
Ours	21.68	31.59	76.29	30.59	42.42	66.30

Euclidean distance between $\mathcal{X}_{0:T-1}$ and $\Xi_{0:T-1}$ over all evaluated points and all time steps, and FDE was computed by averaging the Euclidean distance between $\mathcal{X}_{T-1}$ and Ξ_{T-1} over all evaluated points at the final step. Lower values indicate better performance for both metrics. LTDR followed a delta-based evaluation protocol [21], computed as $\text{LTDR} = \sum_{m=1}^M \sum_{t=0}^{T-1} \delta_m^t / MT$. Here, δ_m^t denotes the fraction of points for which the Euclidean distance between $\mathbf{X}_t$ and Ξ_t was within m pixels at each step t . The LTDR score lies in $[0, 1]$, with higher values indicating better performance, and we set $M = 100$ in this experiment.

These metrics are standard for point tracking [21] and have been adopted for robot flow evaluation [2, 13, 56]. We restricted the evaluation to the end-effector region in each image rather than computing these metrics across the entire image, following prior work [38]. This is because background regions in robot manipulation scenes tend to remain stationary under static camera setups and would dominate the evaluation if included in the metric computation. The end-effector region was identified by displacement-based thresholding and morphological operations, based on the assumption that points corresponding to the end-effector exhibit larger displacements than other regions.

Implementation Details. Our model had approximately 487M trainable parameters. We trained our model on a GeForce RTX 5090 with 32GB of GPU memory. The total training time was approximately 6 hours, and the inference time was approximately 62ms. We used ADE as the validation metric on the validation set of the Bridge V2 dataset, computed every 2,000 steps, and the early stopping strategy was adopted if ADE did not continuously decrease for 20,000 steps. To generate captions for the Narrative Delta loss, we used Qwen3.5-9B [36] and set $N_c = 5$. λ was 0.1 for the first 30k steps and 1.0×10^{-5} thereafter. Following prior work [2, 38], we set $T = 8$, $N = 10 \times 10$, $k = 1$, and $\sigma_0 = 0.05$. Further details are provided in the Supplementary Materials.

4.2 Quantitative Results

Table 2 shows a quantitative comparison between the proposed method and baseline methods on the test sets of Fractal [6] and Bridge V2 [44]. The proposed method achieved the best scores across the metrics on both datasets. Specifically, it achieved ADE scores of 21.68 and 30.59 on Fractal and Bridge V2, respectively, outperforming the best baseline scores of 37.14 and 50.73 by 15.46 and 20.14

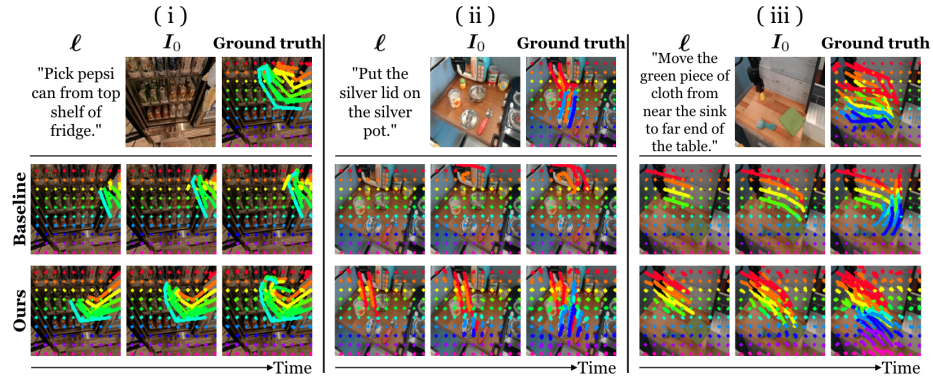


Fig. 3: Qualitative comparison between NarrativeFlow and a baseline method (Im2Flow2Act [49]). In each panel, the first row presents ℓ , I_0 , and the ground-truth robot flow. The second and third rows show the predicted robot flows of the baseline method and the proposed method, respectively.

points. Similarly, it obtained FDE scores of 31.59 and 42.42, compared with the best baseline scores of 47.74 and 68.43, respectively. The performance differences between the proposed and baseline methods were statistically significant ($p < 0.01$).

4.3 Qualitative Results

Fig. 3 shows qualitative results of the proposed method compared with a baseline method, Im2Flow2Act [49]. Fig. 3(i) shows a sample from the Fractal dataset, while Fig. 3(ii) and (iii) show samples from the Bridge V2 dataset. In each panel, the first row presents ℓ , I_0 , and the ground-truth robot flow. The second and third rows show the predicted robot flows of Im2Flow2Act and the proposed method, respectively. They are visualized at intermediate timesteps and the final timestep in temporal order.

Fig. 3(i) shows that the robot picked a Pepsi can from the top shelf of the fridge, where multiple cans of different colors were present in I_0 . Im2Flow2Act failed to yield a robot flow toward the correct can from the phrase "Pepsi can." In contrast, the proposed method generated the robot flow that followed ℓ .

Fig. 3(ii) depicts a manipulation that involved picking the silver lid and then putting it on the silver pot. Im2Flow2Act generated the robot flow that immediately moved toward the pot, without reaching the lid. By contrast, the proposed method distinguished the lid from the pot and successfully generated the robot flow for the two-step manipulation.

In Fig. 3(iii), the robot moved the green cloth from the right side of the table to the left side. Although Im2Flow2Act generated the robot flow that first grasped the correct object, the subsequent motion was in the wrong direction. In contrast, the proposed method generated the robot flow that successfully captured the correct object and moved in the appropriate direction.

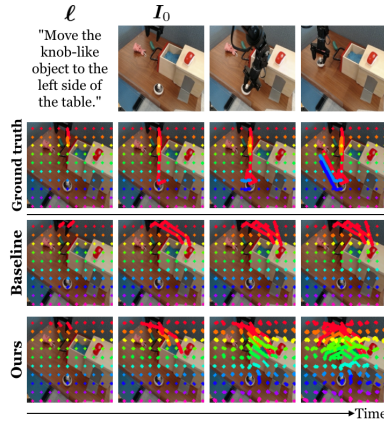


Fig. 4: A failure case of NarrativeFlow on Bridge V2 [44]. The first row shows ℓ , I_0 , and subsequent frames sampled from the ground-truth video in temporal order. The other rows show the robot flows of the ground truth, a baseline (Im2Flow2Act [49]), and the proposed method in temporal order, respectively.

Fig. 4 shows a failure case on Bridge V2. The top row shows ℓ and I_0 , and the following three rows show the robot flows of the ground truth, a baseline (Im2Flow2Act [49]), and the proposed method, respectively. In this case, ℓ was “move the knob-like object to the left side of the table.” The baseline failed to generate the robot flow toward the target object. The proposed method incorrectly generated the flow that targeted the drawer and then opened it. This failure was likely due to the expression “knob-like” in ℓ , which misled the proposed method into identifying the drawer knob as the target object.

4.4 Ablation Studies

Table 3 shows the results of the ablation studies. In these studies, the effects on performance of subtask tokens and the Narrative Delta loss were investigated.

Subtask Token Ablation. To investigate the benefit of using subtask tokens, we evaluated two models against the proposed method. Model (i) did not use any subtask tokens; instead, it used the output tokens of the Transformer encoder to condition the Flow-as-Flow module and to compute the Narrative Delta loss. Instead of two subtask tokens, Model (ii) used a single shared subtask token for both objectives. Model (i) obtained ADE scores that were 1.52 and 2.82 points worse than those of the proposed method on Fractal and Bridge V2, respectively. Similarly, Model (ii) yielded ADE scores 3.04 and 2.61 points worse than those of the proposed method on the two datasets, respectively. These results indicate that the subtask tokens allow each token to specialize for its corresponding objective, improving overall robot flow generation.

Narrative Delta Loss Ablation. To assess the contribution of the Narrative Delta loss, we removed this loss from Model (iii) and trained the model only

Table 3: Quantitative results of ablation studies on the subtask tokens and the Narrative Delta loss. ✓ indicates the use of the Narrative Delta loss. The best scores for each metric are shown in **bold**.

Model	Subtask tokens	Narrative Delta loss	Fractal			Bridge V2		
			ADE ↓	FDE ↓	LTDR ↑[%]	ADE ↓	FDE ↓	LTDR ↑[%]
(i)	0	✓	23.20	32.91	74.84	33.41	47.40	63.42
(ii)	1	✓	24.72	33.96	72.89	33.20	45.30	63.50
(iii)	2		25.83	35.72	71.87	33.79	47.81	63.00
(iv) Ours	2	✓	21.68	31.59	76.29	30.59	42.42	66.30



Fig. 5: The Human Support Robot (HSR) used in our real-world experiments.

with $\mathcal{L}_{\text{FaF}}$. Model (iii) obtained ADE scores that were worse than those of the proposed method by 4.15 and 3.20 points on Fractal and Bridge V2, respectively. This suggests that the Narrative Delta loss enhances robot flow generation by providing an auxiliary supervisory signal for learning task-relevant scene changes.

5 Real-World Experiments

5.1 Experimental Setup

To validate NarrativeFlow in downstream manipulation tasks, we conducted real-world experiments.

Robot Platform. All of the experiments were conducted using the Human Support Robot (HSR), an 11-DoF mobile manipulator developed by Toyota Motor Corporation, as shown in Fig. 5. The visual inputs provided to the policy were captured with an external webcam. Specifically, these inputs consisted of I_0 and the current observation at each timestep during execution.

Tasks. We evaluated NarrativeFlow on three mobile manipulation tasks.

1. **Mobile drawer closing:** In this task, the robot is expected to close an open tabletop drawer by pushing it forward. The generated robot flow must provide the correct pushing direction and an appropriate pushing stroke; otherwise, the drawer cannot be closed completely.
2. **Mobile bin picking:** This task involves picking a specified object from a table, and placing it into a bin. Given ℓ , the model must identify the target object among multiple objects on the table.

Table 4: Quantitative results of real-world experiments. For each method, we report success rates across 20 trials per task, and the average over the three tasks. For the oracle method, the policy was conditioned on the corresponding ground-truth robot flows. The best scores among the proposed and baseline methods are shown in **bold**.

Method [%]	Mobile drawer closing	Mobile bin picking	Mobile cup stacking	Avg.
FLIP [13]	50	25	10	28
Im2Flow2Act [49]	65	45	15	42
Ours	85	60	20	55
Oracle	90	80	20	63

3. **Mobile cup stacking:** This task requires the robot to stack a cup onto two nested cups. The generated robot flow must support precise alignment of the small cups; even a misalignment of a few centimeters can result in failure.

Mobile manipulation is more challenging for a flow-conditioned policy than table-top manipulation because the policy is required to map the generated robot flows to both end-effector and mobile-base actions.

Baselines and Policy. We used two baseline methods, Im2Flow2Act [49] and FLIP [13], which are the same baselines evaluated in Sec. 4. For manipulation, we used a flow-conditioned policy following the DiT-based architecture and training setup in [38]. We collected an average of 30 teleoperated demonstrations for each task, and fine-tuned each model on the collected data.

5.2 Quantitative Results

Table 4 shows the quantitative results of the real-world experiments. For each method, we report success rates across 20 trials per task, along with the average rate over the three tasks. The table shows NarrativeFlow achieved higher success rates than both baseline methods across the three tasks. Overall, NarrativeFlow achieved an average success rate of 55%, outperforming the strongest baseline, Im2Flow2Act [49], by 13 points. We also report the success rates of the oracle method, in which the policy was conditioned on the corresponding ground-truth robot flows. The average success-rate gap relative to the oracle method was 8 points for NarrativeFlow, whereas it was 21 points for Im2Flow2Act. This smaller gap suggests that the robot flows generated by NarrativeFlow are more effective for the downstream manipulation policy than those of the baseline methods.

5.3 Qualitative Results

Fig. 6 shows the qualitative results of real-world experiments. In each panel, the top row shows ℓ , I_0 , and the generated robot flow, while the bottom row shows the corresponding execution in temporal order.

Fig. 6(i) depicts a successful execution of the **mobile drawer closing** task. In this case, the generated robot flow provided the correct pushing direction

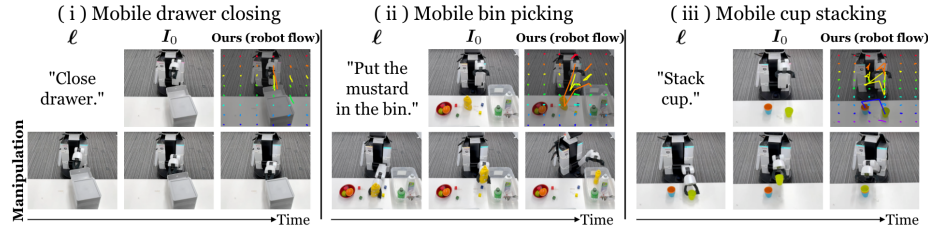


Fig. 6: Qualitative results of real-world experiments. The panels show successful executions of three manipulation tasks: (i) **mobile drawer closing**, (ii) **mobile bin picking**, and (iii) **mobile cup stacking**. In each panel, the top row shows ℓ , I_0 , and the generated robot flow, while the bottom row shows the downstream manipulation conditioned on the robot flow.

and an appropriate pushing stroke, enabling the robot to completely close the drawer.

Fig. 6(ii) presents a **mobile bin picking** case, where the robot should grasp the target object and place it into the designated container. The proposed method correctly identified the target object specified by ℓ , and the robot completed the task using the generated robot flow.

In Fig. 6(iii), the robot successfully executed the **mobile cup stacking** task. The generated robot flow was aligned with the positions of the target cup and the nested cups, guiding the robot to correctly perform the manipulation.

6 Conclusion

In this study, we focused on language-conditioned flow-based manipulation. The contributions of this study are as follows:

1. We proposed NarrativeFlow, a language-conditioned robot flow generation method that formulates robot flows as probability velocity fields within a flow-matching framework, enabling high-quality robot flow generation.
2. We introduced the Narrative Delta loss as an auxiliary objective that aligns the model's intermediate representations with task-relevant scene changes. We further introduced the subtask tokens to mitigate interference between this auxiliary objective and the primary flow-generation objective.
3. NarrativeFlow outperformed the baseline methods across standard datasets.
4. Through real-world experiments, NarrativeFlow achieved higher success rates than the baseline methods across multiple manipulation tasks.

Future work will include lifting 2D image-plane robot flows into 3D space by incorporating depth information. This could enable depth-wise manipulation, such as pulling a book straight out of a bookshelf toward the camera viewpoint. In this case, the book appears nearly static in the image plane, resulting in a negligible 2D flow despite substantial motion in 3D.

Acknowledgments. This work was partially supported by JST CREST and JSPS Fellows Grant Number JP26KJ1969.

References

1. Bai, S., Chen, K., Liu, X., Wang, J., Song, S., Dang, K., Wang, P., Wang, S., et al.: Qwen2.5-VL Technical Report. arXiv preprint arXiv:2502.13923 (2025)
2. Bharadhwaj, H., Mottaghi, R., Gupta, A., Tulsiani, S.: Track2Act: Predicting Point Tracks from Internet Videos enables Generalizable Robot Manipulation. In: ECCV. pp. 306–324 (2024)
3. Black, K., Brown, N., Darpinian, J., Dhabalia, K., Driess, D., Esmail, A., Equi, M., Finn, C., Fusai, N., Galliker, M., et al.: $\pi_{0.5}$: a Vision-Language-Action Model with Open-World Generalization. In: CoRL. pp. 17–40 (2025)
4. Black, K., Brown, N., Driess, D., Esmail, A., Equi, M., Finn, C., Fusai, N., Groom, L., Hausman, K., Ichter, B., Jakubczak, S., Jones, T., et al.: π_0 : A Vision-Language-Action Flow Model for General Robot Control. In: RSS (2025)
5. Black, K., Nakamoto, M., Atreya, P., et al.: Zero-Shot Robotic Manipulation with Pretrained Image-Editing Diffusion Models. In: ICLR (2024)
6. Brohan, A., Brown, N., Carbajal, J., Chebotar, Y., Dabis, J., et al.: RT-1: Robotics Transformer for Real-World Control at Scale. In: RSS (2023)
7. Bu, Q., Cai, J., Chen, L., Cui, X., Ding, Y., Feng, S., He, X., Huang, X., Jiang, S., Jiang, Y., et al.: AgiBot World Colosseo: A Large-scale Manipulation Platform for Scalable and Intelligent Embodied Systems. In: IROS. pp. 3549–3556 (2025)
8. Chen, H., Sun, B., Zhang, A., Pollefeys, M., Leutenegger, S.: VidBot: Learning Generalizable 3D Actions from In-the-Wild 2D Human Videos for Zero-Shot Robotic Manipulation. In: CVPR. pp. 27661–27672 (2025)
9. Chen, T., Mu, Y., Liang, Z., Chen, Z., Peng, S., Chen, Q., Xu, M., Hu, R., Zhang, H., Li, X., Luo, P.: G3Flow: Generative 3D Semantic Flow for Pose-aware and Generalizable Object Manipulation. In: CVPR. pp. 1735–1744 (2025)
10. Chen, Y.C., Li, L., Yu, L., El Kholy, A., Ahmed, F., Gan, Z., et al.: UNITER: UNiversal Image-TExt Representation Learning. In: ECCV. pp. 104–120 (2020)
11. Chen, Y., Li, P., Huang, Y., Yang, J., Chen, K., Wang, L.: EC-Flow: Enabling Versatile Robotic Manipulation from Action-Unlabeled Videos via Embodiment-Centric Flow. In: ICCV. pp. 11958–11968 (2025)
12. Deng, Y., Mo, K., Xia, C., Wang, X.: Learning Language-Conditioned Deformable Object Manipulation with Graph Dynamics. In: ICRA. pp. 7508–7514 (2024)
13. Gao, C., Zhang, H., et al.: FLIP: Flow-Centric Generative Planning as General-Purpose Manipulation World Model. In: ICLR (2025)
14. Goko, M., et al.: Task Success Prediction for Open-Vocabulary Manipulation Based on Multi-Level Aligned Representations. In: CoRL. pp. 3242–3263 (2024)
15. He, K., Zhang, X., Ren, S., Sun, J.: Deep Residual Learning for Image Recognition. In: CVPR. pp. 770–778 (2016)
16. Hong, W., Cheng, Y., Yang, Z., Wang, W., Wang, L., Gu, X., Huang, S., Dong, Y., Tang, J.: MotionBench: Benchmarking and Improving Fine-grained Video Motion Understanding for Vision Language Models. In: CVPR. pp. 8450–8460 (2025)
17. Huang, W., Chao, Y.W., et al.: PointWorld: Scaling 3D World Models for In-The-Wild Robotic Manipulation. In: CVPR. pp. 20765–20779 (2026)
18. Ji, Y., Tan, H., Shi, J., Hao, X., Zhang, Y., Zhang, H., Wang, P., Zhao, M., Mu, Y., An, P., et al.: RoboBrain: A Unified Brain Model for Robotic Manipulation from Abstract to Concrete. In: CVPR. pp. 1724–1734 (2025)
19. Kambara, M., et al.: LILAC: Language-Conditioned Object-Centric Optical Flow for Open-Loop Trajectory Generation. IEEE RA-L **11**(6), 6767–6774 (2026)

20. Karaev, N., Makarov, Y., Wang, J., et al.: CoTracker3: Simpler and Better Point Tracking by Pseudo-Labeling Real Videos. In: ICCV. pp. 6013–6022 (2025)
21. Karaev, N., Rocco, I., Graham, B., Neverova, N., Vedaldi, A., Rupprecht, C.: CoTracker: It is Better to Track Together. In: ECCV. pp. 18–35 (2024)
22. Kawaharazuka, K., et al.: Vision-Language-Action Models for Robotics: A Review Towards Real-World Applications. *IEEE Access* **13**, 162467–162504 (2025)
23. Khazatsky, A., Pertsch, K., Nair, S., Balakrishna, A., Dasari, S., Karamcheti, S., Nasiriany, S., Srirama, M., Chen, L., Ellis, K., Fagan, P., Hejna, J., et al.: DROID: A Large-Scale In-The-Wild Robot Manipulation Dataset. In: RSS (2024)
24. Kim, M.J., Pertsch, K., Karamcheti, S., Xiao, T., Balakrishna, A., Nair, S., Rafailov, R., Foster, E., Sanketi, P., Vuong, Q., et al.: OpenVLA: An Open-Source Vision-Language-Action Model. In: CoRL. pp. 2679–2713 (2024)
25. Krishna, S., Eisner, B., Zhan, H., Yuan, Y., Zhen, H., et al.: GHOST: Hierarchical Sub-Goal Policies for Generalizing Robot Manipulation. In: RSS (2026)
26. Li, Z., Geng, W., et al.: Skip-Plan: Procedure Planning in Instructional Videos via Condensed Action Space Learning. In: ICCV. pp. 10297–10306 (2023)
27. Lin, M., Ding, P., Wang, S., Zhuang, Z., Liu, Y., Tong, X., Song, W., Lyu, S., et al.: HiF-VLA: Hindsight, Insight and Foresight through Motion Representation for Vision-Language-Action Models. In: CVPR. pp. 20732–20742 (2026)
28. Lin, T., Zhong, Y., Du, Y., Zhang, J., Liu, J., Chen, Y., Gu, E., Liu, Z., Cai, H., Zou, Y., Zou, L., Zhou, Z., et al.: Evo-1: Lightweight Vision-Language-Action Model with Preserved Semantic Alignment. In: CVPR. pp. 13397–13406 (2026)
29. Lipman, Y., et al.: Flow Matching for Generative Modeling. In: ICLR (2023)
30. Myers, V., He, W., Fang, K., Walke, R., Hansen-Estruch, P., Cheng, C.A., Jalobeanu, M., et al.: Goal Representations for Instruction Following: A Semi-Supervised Language Interface to Control. In: CoRL. pp. 3894–3908 (2023)
31. Nagasinghe, Y., Zhou, H., Gunawardhana, M., Min, R., Harari, D., Khan, H.: Why Not Use Your Textbook? Knowledge-Enhanced Procedure Planning of Instructional Videos. In: CVPR. pp. 18816–18826 (2024)
32. Niu, Y., Guo, W., Chen, L., Lin, X., Chang, S.F.: SCHEMA: State CHangEs MATter for Procedure Planning in Instructional Videos. In: ICLR (2024)
33. O’Neill, A., Rehman, A., Maddukuri, A., Gupta, A., Padalkar, A., Lee, A., Pooley, A., Gupta, A., Mandlekar, A., Jain, A., et al.: Open X-Embodiment: Robotic Learning Datasets and RT-X Models. In: ICRA. pp. 6892–6903 (2024)
34. Peebles, W., Xie, S.: Scalable Diffusion Models with Transformers. In: ICCV. pp. 4172–4182 (2023)
35. Qu, D., Song, H., Chen, Q., Yao, Y., Ye, X., Ding, Y., et al.: SpatialVLA: Exploring Spatial Representations for Visual-Language-Action Models. In: RSS (2025)
36. Qwen Team: Qwen3.5: Towards Native Multimodal Agents (2026), <https://qwen.ai/blog?id=qwen3.5>
37. Routray, S., Pan, H., Jain, U., Bahl, S., Pathak, D.: ViPRA: Video Prediction for Robot Actions. In: ICLR (2026)
38. Seno, K., Yashima, D., Takagi, Y., Tokura, K., Sugiura, K.: Flow as Flow: Modeling Robot Velocity Fields as Probability Velocity Fields for Flow-Based Object Manipulation. *arXiv preprint arXiv:2606.23090* (2026)
39. Siméoni, O., Vo, V., Seitzer, M., Baldassarre, F., Oquab, M., Jose, C., Khalidov, V., Szafraniec, M., Yi, S., Ramamonjisoa, M., Massa, F., Haziza, D., Wehrstedt, L., Wang, J., et al.: DINOv3. *arXiv preprint arXiv:2508.10104* (2025)
40. Sundaresan, P., Vuong, Q., Gu, J., Xu, P., Xiao, T., Kirmani, S., Yu, T., Stark, M., Jain, A., Hausman, K., Sadigh, D., et al.: RT-Sketch: Goal-Conditioned Imitation Learning from Hand-Drawn Sketches. In: CoRL. pp. 70–96 (2024)

41. Tan, H., Bansal, M.: LXMERT: Learning Cross-Modality Encoder Representations from Transformers. In: EMNLP-IJCNLP. pp. 5100–5111 (2019)
42. Tran, P.V.: SSLayout360: Semi-Supervised Indoor Layout Estimation From 360deg Panorama. In: CVPR. pp. 15353–15362 (2021)
43. Vecerik, M., Doersch, C., Yang, Y., Davchev, T., et al.: RoboTAP: Tracking Arbitrary Points for Few-Shot Visual Imitation. In: ICRA. pp. 5397–5403 (2024)
44. Walke, H., Black, K., Lee, A., Kim, M.J., Du, M., Zheng, C., Zhao, T., Hansen-Estruch, P., Vuong, Q., He, A., Myers, V., et al.: BridgeData V2: A Dataset for Robot Learning at Scale. In: CoRL. pp. 1723–1736 (2023)
45. Wang, H., Wu, Y., Guo, S., Wang, L.: PDPP: Projected Diffusion for Procedure Planning in Instructional Videos. In: CVPR. pp. 14836–14845 (2023)
46. Wang, Z., Zheng, H., Nie, Y., Xu, W., Wang, Q., Ye, H., Li, Z., Zhang, K., Cheng, X., et al.: All Robots in One: A New Standard and Unified Dataset for Versatile, General-Purpose Embodied Agents. arXiv preprint arXiv:2408.10899 (2024)
47. Wen, C., Lin, X., So, J., Chen, K., Dou, Q., Gao, Y., Abbeel, P.: Any-point Trajectory Modeling for Policy Learning. In: RSS (2024)
48. Wu, K., Hou, C., Liu, J., Che, Z., Ju, X., Yang, Z., Li, M., Zhao, Y., Xu, Z., Yang, G., Fan, S., Wang, X., et al.: RoboMIND: Benchmark on Multi-embodiment Intelligence Normative Data for Robot Manipulation. In: RSS (2025)
49. Xu, M., Xu, Z., Xu, Y., Chi, C., et al.: Flow as the Cross-Domain Manipulation Interface. In: CoRL. pp. 2475–2499 (2024)
50. Xue, Z., An, J., Yang, X., Grauman, K.: Progress-Aware Video Frame Captioning. In: CVPR. pp. 13639–13650 (2025)
51. Xue, Z., Ashutosh, K., Grauman, K.: Learning Object State Changes in Videos: An Open-World Perspective. In: CVPR. pp. 18493–18503 (2024)
52. Yang, J., et al.: Tra-MoE: Learning Trajectory Prediction Model from Multiple Domains for Adaptive Policy Conditioning. In: CVPR. pp. 6960–6970 (2025)
53. Yang, S., Li, H., Wang, B., Chen, Y., Tian, Y., et al.: Vision-Language-Action Instruction Tuning: From Understanding to Manipulation. In: ICLR (2026)
54. Yang, Y., Cai, Z., Tian, Y., Zeng, J., Pang, J.: Gripper Keypose and Object Point-flow as Interfaces for Bimanual Robotic Manipulation. In: RSS (2025)
55. Yashima, D., Kurita, S., Oda, Y., Sugiura, K.: ReMoRa: Multimodal Large Language Model based on Refined Motion Representation for Long-Video Understanding. In: CVPR. pp. 31845–31855 (2026)
56. Yoshida, T., et al.: Generating 6DoF Object Manipulation Trajectories from Action Description in Egocentric Vision. In: CVPR. pp. 17370–17382 (2025)
57. Yuan, C., Joshi, S., Zhu, S., Su, H., Zhao, H., Gao, Y.: RoboEngine: Plug-and-Play Robot Data Augmentation with Semantic Robot Segmentation and Background Generation. In: IROS. pp. 7622–7629 (2025)
58. Yuan, C., Wen, C., Zhang, T., Gao, Y.: General Flow as Foundation Affordance for Scalable Robot Learning. In: CoRL. pp. 1541–1566 (2024)
59. Zhang, H., et al.: FlowBot++: Learning Generalized Articulated Objects Manipulation via Articulation Projection. In: CoRL. pp. 1222–1241 (2023)
60. Zhang, Y., Li, M., Long, D., Zhang, X., Lin, H., Yang, B., Xie, P., Yang, A., Liu, D., Lin, J., Huang, F., Zhou, J.: Qwen3 Embedding: Advancing Text Embedding and Reranking Through Foundation Models. arXiv preprint arXiv:2506.05176 (2025)
61. Zhao, Q., Lu, Y., Kim, J., Fu, Z., Zhang, Z., Wu, Y., Li, Z., Ma, Q., Han, S., Finn, C., Handa, A., Liu, M.Y., Xiang, D., et al.: CoT-VLA: Visual Chain-of-Thought Reasoning for Vision-Language-Action Models. In: CVPR. pp. 1702–1713 (2025)

62. Zhong, Y., Bai, F., Cai, S., Huang, X., Chen, Z., Zhang, X., Wang, Y., Guo, S., Guan, T., Lui, K.N., Qi, Z., et al.: A Survey on Vision-Language-Action Models: An Action Tokenization Perspective. arXiv preprint arXiv:2507.01925 (2025)

NarrativeFlow: Flow-Based Vision-Language-Action Model Using Robot Velocity Fields (Supplementary Material)

Shota Kobayashi, Koki Seno, Daichi Yashima, and Komei Sugiura

Keio University, Japan

{shotakoba10267,koki.seno,ydaichi1207,komei.sugiura}@keio.jp

A Additional Related Work

Standard Datasets in Robot Learning. Real-world robot manipulation datasets have substantially expanded in both scale and embodiment diversity. Open X-Embodiment (OXE) [33] is a large-scale cross-embodiment dataset constructed by aggregating demonstration data from heterogeneous platforms, and it has been widely used for training vision-language-action (VLA) models (e.g., [3, 4, 24, 35]). Among the datasets within OXE, Fractal [6] and Bridge V2 [44] provide language-annotated demonstrations for diverse manipulation tasks. Complementing these datasets, DROID [23] provides large-scale manipulation demonstrations across diverse in-the-wild settings, including households and offices. Recent efforts have further expanded the range of robot embodiments, with AgiBot World [7], RoboMIND [48], and ARIO [46] covering diverse platforms, including humanoid robots.

B Implementation Details

Table A shows the training settings of the proposed method. In terms of the architecture, we used four Transformer layers in the vision-language fusion encoder. As specified in Sec. 4.1, we set λ to a high value for the early training steps, and to a low value for the remaining steps [37, 42]. This schedule assigns a larger weight to $\mathcal{L}_{\text{ND}}$ in the initial phase, grounding the model’s intermediate representations in task-relevant scene changes. In the subsequent phase, the training objective focuses on robot flow generation based on the learned scene-change prior.

In addition, for the caption generation used in the Narrative Delta loss (Sec. 3.4), we used the prompt shown below with a maximum token count M . We set $M = 32$ based on preliminary caption-generation trials without a length constraint, where the generated captions were observed to fit within 32 tokens.

Table A: Training settings of the proposed method.

Optimizer	AdamW
Batch size	128
Learning rate	1.0×10^{-4}
LR schedule	Cosine decay
Warmup steps	9,000
Training steps	300,000
Weight decay	0.01
EMA decay	0.9999

Full prompt for the Narrative Delta loss

You are given two images from the same robot manipulation episode. The first image is the initial frame, and the second image is the final frame. Describe only the meaningful visual difference between the initial and final frames. Focus on object motion, hand or robot motion, state changes, spatial relations, and changes caused by the action. Do not mention unchanged objects or background unless needed to clarify the change. Write $\{N_c\}$ non-duplicate descriptions for these same two frames. Each description should use different wording and emphasize a different meaningful aspect of the visual change when possible. Each description must be at most $\{M\}$ tokens. Each description must be one short sentence. Return only a valid JSON object. The keys must be description ids from 0 to $\{N_c - 1\}$, and each value must be one concise English sentence. Do not explain your reasoning. Do not write drafts or analysis. Do not include markdown fences or any extra text. Only produce the final JSON object.

C Preliminary Analysis of Narrative Delta Loss

Table B shows the results of our preliminary analysis of the Narrative Delta loss. We evaluated three design choices: the number of captions, the loss function, and the source for caption generation.

C.1 Number of Captions

To investigate the effect of the number of captions used in the Narrative Delta loss, we compared models with N_c set to 1, 3, and 5, corresponding to Models (i), (ii), and (v), respectively. Following the main experimental settings, each training sample was associated with five captions generated using the prompt in Sec. B. In this analysis, each model used the first N_c captions from the five captions.

As shown in Table B, the proposed model with $N_c = 5$ achieved the best scores across the metrics on both datasets. In particular, on Bridge V2, increasing

Table B: Quantitative results of preliminary analysis of the Narrative Delta loss. $\mathbf{I}_0/\mathbf{G}$ indicates that captions were generated from $\mathbf{I}_0$ and $\mathbf{G}$, while Video indicates that captions were generated from multiple frames sampled from the video clip. The best scores for each metric are shown in **bold**.

Model	N_c	Loss function	Caption source	Fractal			Bridge V2		
				ADE ↓	FDE ↓	LTDR ↑[%]	ADE ↓	FDE ↓	LTDR ↑[%]
(i)	1	Cosine similarity	$\mathbf{I}_0/\mathbf{G}$	21.70	31.68	76.26	31.24	43.64	65.42
(ii)	3	Cosine similarity	$\mathbf{I}_0/\mathbf{G}$	21.71	31.68	76.24	30.63	42.80	66.09
(iii)	5	L2	$\mathbf{I}_0/\mathbf{G}$	27.00	36.26	70.36	34.98	48.28	61.69
(iv)	5	Cosine similarity	Video	22.26	31.76	75.63	32.39	45.60	64.70
(v) Ours	5	Cosine similarity	$\mathbf{I}_0/\mathbf{G}$	21.68	31.59	76.29	30.59	42.42	66.30

N_c yielded better scores across all three metrics. These results suggest that using multiple captions is effective for the Narrative Delta loss, since it helps the model capture task-relevant semantics that are shared across different captions of the same scene change.

C.2 Loss Function

To assess the effect of the loss function used for the Narrative Delta loss, we compared Model (iii) with the proposed model. Model (iii) replaces the cosine similarity loss in Eq. (1) with the following L2 loss:

$$\mathcal{L}'_{\text{ND}} = \frac{1}{N_c} \sum_{i=1}^{N_c} \|\mathbf{z}_{\text{st}}^{(2)} - \mathbf{z}_{\text{ND}}^{(i)}\|_2^2. \quad (1)$$

As shown in Table B, Model (iii) yielded worse scores than the proposed model across the three metrics on both datasets. Since Qwen3-Embedding-0.6B used in our method (Sec. 3.4) is trained with an InfoNCE-based contrastive objective with cosine similarity [60], the Narrative Delta loss with cosine similarity is expected to be more consistent with the learned embedding space.

C.3 Source for Caption Generation

We also evaluated Model (iv), which uses captions generated from video clips for the Narrative Delta loss, instead of captions generated from $\mathbf{I}_0$ and $\mathbf{G}$. Specifically, for each video clip, we uniformly selected 8 frames (following T in Sec. 4.1) from the segment between $\mathbf{I}_0$ and $\mathbf{G}$, including both endpoints. We then provided the MLLM with these frames and the video-captioning prompt shown below, with $N_c = 5$ and $M = 32$ following Sec. B.

Table B shows that Model (iv) performed worse than the proposed method on both datasets. In particular, Model (iv) yielded worse ADE scores than the proposed method by 0.58 on Fractal and 1.80 on Bridge V2. This result indicates that captions generated from $\mathbf{I}_0$ and $\mathbf{G}$ provide a more effective training signal

for robot flow generation. We attribute this to two factors. First, video captions tend to emphasize aspects of the task-relevant motion that are easy to describe, rather than faithfully reflecting the full trajectory (e.g., [16]). Second, precise trajectory information is already supervised by $\mathcal{L}_{\text{FaF}}$ through $\Xi_{0:T-1}$; encoding it again as a coarse text-level embedding therefore adds no effective constraint.

Full prompt to generate captions from videos

You are given frames sampled chronologically from a robot manipulation video. Describe the primary action visible across the frames. Where relevant, include the manipulated object’s initial position, trajectory, and final position, as well as any meaningful state changes and notable changes in object shape, configuration, or spatial relationships. Use clear, natural spatial language grounded in what is visible, drawing on directional and positional terms. Keep descriptions concise but specific, and omit background elements that remain unchanged throughout.

Do not explain your reasoning or produce any intermediate analysis.

Write $\{N_c\}$ non-duplicate descriptions of the video. Vary the wording across descriptions: use different expressions for positions, regions, and state changes rather than repeating the same phrasing. Each description must be a single short sentence within $\{M\}$ tokens. Return a valid JSON object with integer keys from 0 to $\{N_c - 1\}$, each mapping to one description string. Do not include markdown fences or any surrounding text. Output only the final JSON object.

D Additional Experiments

D.1 Additional Ablation Studies

To further examine the effects of MLLM size for caption generation and continuous velocity-field modeling in the proposed method, we conducted two additional ablation studies. Table C shows the results of these studies.

MLLM Size for Caption Generation. To assess the effect of the MLLM size for caption generation, we evaluated Model (i) using Qwen3.5-0.8B [36], a smaller MLLM than Qwen3.5-9B [36] used in the proposed method. Table C shows that Model (i) underperformed the proposed method across the three metrics on both datasets. This result indicates that captions generated by a larger MLLM lead to more effective auxiliary supervision through the Narrative Delta loss, improving robot flow generation. Nevertheless, Model (i) outperformed the baseline methods in Table 2 across the metrics. This suggests that even with a small MLLM for caption generation, the proposed method remains sufficiently effective for robot flow generation.

Continuous Velocity-Field Modeling. To investigate the contribution of continuous velocity-field modeling in the language-conditioned setting, we evaluated Model (ii), which generated robot flows as displacements of sparse keypoints by setting $\sigma_0 = 0$. In this case, since sampling $\mathbf{X}$ deterministically yields Ξ_i , robot

Table C: Quantitative results of the additional ablation studies.

Model	Caption generator	σ_0	Fractal			Bridge V2		
			ADE↓	FDE↓	LTDR↑[%]	ADE↓	FDE↓	LTDR↑[%]
(i)	Qwen3.5-0.8B	0.05	26.40	36.59	71.07	32.72	44.60	63.89
(ii)	Qwen3.5-9B	0	26.43	36.83	70.86	34.35	48.01	62.28
(iii) Ours	Qwen3.5-9B	0.05	21.68	31.59	76.29	30.59	42.42	66.30

Table D: Additional quantitative results on real-world experiments across 13 manipulation tasks.

Method [%]	Mobile bin pushing	Mobile chair pushing	Mobile drawer closing	Mobile box closing	Mobile towel taking	Mobile bin picking	Mobile object placing	Mobile table bussing	Mobile laptop closing	Mobile water pouring	Mobile drawer opening	Mobile cup stacking	Mobile block stacking	Avg.
FLIP [13]	45	55	50	60	50	25	15	15	25	5	10	10	5	28
Im2Flow2Act [49]	70	70	65	65	60	45	50	40	45	20	20	15	20	45
Ours	90	90	85	85	70	60	65	70	50	35	20	20	20	58

flows are not modeled as dense velocity fields. Table C shows that the proposed method outperformed Model (ii) across the three metrics on both datasets. This result indicates that continuous velocity-field modeling is effective for language-conditioned robot flow generation.

D.2 Additional Real-world Experiments

To further evaluate the proposed method in real-world mobile manipulation beyond the three tasks in Table 4, we conducted experiments on 10 additional tasks. Table D shows the quantitative results for a total of 13 tasks, with 260 trials per method. The proposed method achieved a higher average success rate than the baseline methods. These results indicate that robot flows generated by the proposed method are effective across a broader range of real-world mobile manipulation tasks.

E Error Analysis

We define failure cases as samples in which the proposed method yielded worse ADE scores than Im2Flow2Act. There were 185 and 290 failure cases on Fractal and Bridge V2, respectively. We conducted an error analysis of 100 of these cases sampled from Bridge V2, and the results are listed in Table E. These cases can be broadly grouped into the following six categories:

- **Multimodal language comprehension errors:** This refers to cases where the model incorrectly interpreted ℓ and $\mathbf{I}_0$; for example, misunderstanding the target object. A representative example was a case in which the model generated $\mathcal{X}_{0:T-1}$ that corresponded to grasping an eggplant, although ℓ was “place the spatula just behind the eggplant.”

Table E: Categorization of failure cases.

Error category	# Errors
Multimodal language comprehension errors	30
Intermediate trajectory deviations	22
Terminal position errors	14
Ambiguous language instructions	9
Target object occlusion	8
Annotation errors	17
Total	100

- **Intermediate trajectory deviations:** This category includes cases in which the intermediate trajectories of $\mathcal{X}_{0:T-1}$ deviated substantially from those of $\Xi_{0:T-1}$, while $\mathcal{X}_{0:T-1}$ corresponded to manipulating the correct object toward the appropriate target location.
- **Terminal position errors:** This category refers to cases in which the terminal position of the target object deviated from the ground-truth terminal position, while the intermediate trajectories of $\mathcal{X}_{0:T-1}$ remained close to those of $\Xi_{0:T-1}$. As in the **intermediate trajectory deviations** category, the target object was correctly identified and manipulated.
- **Ambiguous language instructions:** This category includes cases in which ℓ was ambiguous. For example, when two block towers were visible in I_0 and ℓ was “move the block from one tower to the other,” ℓ did not uniquely identify which block should be manipulated. This ambiguity left multiple candidate objects equally plausible; consequently, some degree of randomness in object selection was unavoidable.
- **Target object occlusion:** This category refers to cases in which the target object was heavily occluded in I_0 . This lack of visibility prevented the model from identifying it.
- **Annotation errors:** This category covers cases in which either $\Xi_{0:T-1}$ or ℓ was unreliable. These cases involved tracking failures in CoTracker3 [20] or mismatches between ℓ and the manipulation in the source demonstration. Among the 17 cases in this category, 13 involved errors in the human annotation of ℓ in the Bridge V2 dataset.

Table E shows that the main challenge was multimodal language comprehension. A possible solution is to introduce an object-centric region proposal module, which first extracts candidate object regions from I_0 and then selects the region most relevant to ℓ before robot flow generation. By explicitly identifying the target object, this module is expected to mitigate failures caused by incorrect target-object identification.